

Tour Planner – Protocol

Erion Polisi

Sebastian Anderseka

Inhalt

1	Project Overview	3
2	Technical Decisions, Failures and Selected Solutions	4
2.1	State management — Signals over RxJS	4
2.2	MVVM split	4
2.3	Modal coordination — Facade	4
2.4	Routing — Adapter for external map APIs	4
2.5	Search UX — scope-aware global search	5
2.6	Responsive layout — reusable layout service	5
2.7	Backend status	5
3	Design Patterns Implemented	6
4	Use-Case Diagram	7
5	UI-Flow Wireframes	8
6	Architecture — Class Diagram	9
7	Sequence Diagram — Full-text Search on Tours	10
8	Time Tracking	11
9	Repository MapTour	13

1 Project Overview

The Tour Planner is a fullstack web application that lets users plan tours (walking, cycling, driving), pick routes interactively on a map, and keep a journal of completed tours through structured tour logs. The system is split into an **ASP.NET Core Web API** (.NET 10) and an **Angular 21** single-page application with server-side rendering, styled with Tailwind CSS.

Frontend	Angular 21.2, TypeScript 5.9, Tailwind CSS 4.2, Lucide icons, Leaflet 1.9
Backend	ASP.NET Core (.NET 10), OpenAPI
External APIs	Nominatim (geocoding), OSRM (routing)
Architecture	MVVM, reactive Angular signals, single-page app + SSR
Tooling	Angular CLI 21.2, Vitest, Git/GitHub, Jira (SCRUM-prefixed branches)

2 Technical Decisions, Failures and Selected Solutions

2.1 State management — Signals over RxJS

Decision. All shared state is exposed as Angular **signals** plus `computed()` derivations. RxJS is only present transitively (Angular bundles it) and is not used in any feature flow.

Why. Signals deliver synchronous, fine-grained change detection without subscription bookkeeping. With `computed()` the search filter, the difficulty average and the dashboard widgets all derive automatically from the underlying service signals. This kept ViewModels shorthand removed an entire class of unsubscribe bugs.

Failure case. A first draft of the map picker emitted route results through an EventEmitter plus a manual `OnChanges`. Coordinates and route distance went out of sync whenever the user typed in the search bar before a route was computed. The fix was to lift the picker's mutable state into `MapPickerViewModel` and expose `coords`, `routeResult` and `loading` as signals, so the create-tour form is always a pure consumer.

2.2 MVVM split

Decision. Each non-trivial component gets its own `@Injectable()` ViewModel (not provided in: 'root'), provided in the component metadata. Components are thin: they wire(click) to a VM method and bind to VM signals.

Why. Logic is independently testable. Components stay declarative HTML. Two early components (tour-card, tour-list) initially mixed filtering, sorting and modal opening with the template. The MVVM refactoring commits (cc2026b, 9f7f37c, 47d1529, b50aa0e) extracted that into `TourListViewModel` and `TourDetailViewModel`.

2.3 Modal coordination — Facade

Decision. A single `ModalService` owns all modal state (`activeTour`, `activeLog`, `editMode`, `createTourOpen`, `addLogOpen`). Components call e.g. `modalService.openAddLog()`.

Why. Several screens (Dashboard, Tours, Logs) open the same modals. Without a facade each page would have its own boolean signal and the modal components would have to be duplicated per page. Putting all four modals in `MainLayout` and toggling them via `ModalService` makes the modals reusable and avoids overlapping outlets.

2.4 Routing — Adapter for external map APIs

Decision. Two thin adapters wrap external HTTP APIs:

- `NominatimService` → `nominatim.openstreetmap.org` (geocoding and reverse geocoding)

- OsmRoutingService → router.project-osrm.org (driving/cycling/walking profiles)

Both return strongly-typed objects from models/geo.model.ts (SearchResult, RouteResult).

Why. The map picker only knows the domain types; replacing OSRM with another routing provider would only require swapping the adapter implementation. The transportType enum (walking | cycling | driving) is used as a **Strategy** to pick the OSRM profile inside the adapter.

2.5 Search UX — scope-aware global search

Initial design. A single global query signal in SearchService.query. The navbar always showed “Search tours...”, even on the logs page; the query persisted across route changes and there was no way to clear it.

Problems detected on the logs page. Switching from Tours to Logs kept the previous filter active and the placeholder lied about what was being searched.

Selected solution. SearchService now exposes: `readonly scope = signal<'tours' | 'logs' | null>(null);` `readonly placeholder = computed(() => ...);` `setScope(scope) { this.scope.set(scope); this.query.set(""); }`

ToursPage and LogsPage call `setScope()` in `ngOnInit` and reset to null in `ngOnDestroy`. The navbar input is disabled when the scope is null (Dashboard). The page headers show a match-count chip (3 of 12 match) and the empty state differentiates between “nothing yet” and “no matches for X”.

2.6 Responsive layout — reusable layout service

Initial design. Fixed widths (w-72 sidebar, w-96 search input), no media queries. On a phone the sidebar consumed the full screen and the search field overflowed.

Selected solution. A new reusable LayoutService exposes `sidebarOpen` and `toggleSidebar()/closeSidebar()`. The navbar renders a hamburger button (lg:hidden), the sidebar becomes a translated off-canvas drawer below the lg breakpoint, and a backdrop in MainLayout closes it on tap. All modal grids were changed from `grid-cols-2` to `grid-cols-1 sm:grid-cols-2`, page padding from `p-8` to `p-4 md:p-8`, and headers from `flex items-center justify-between` to `flex flex-col sm:flex-row`

2.7 Backend status

Current state. The .NET 10 API is bootstrapped with OpenAPI but does not yet expose endpoints; the frontend uses signal-backed in-memory services for tours and logs.

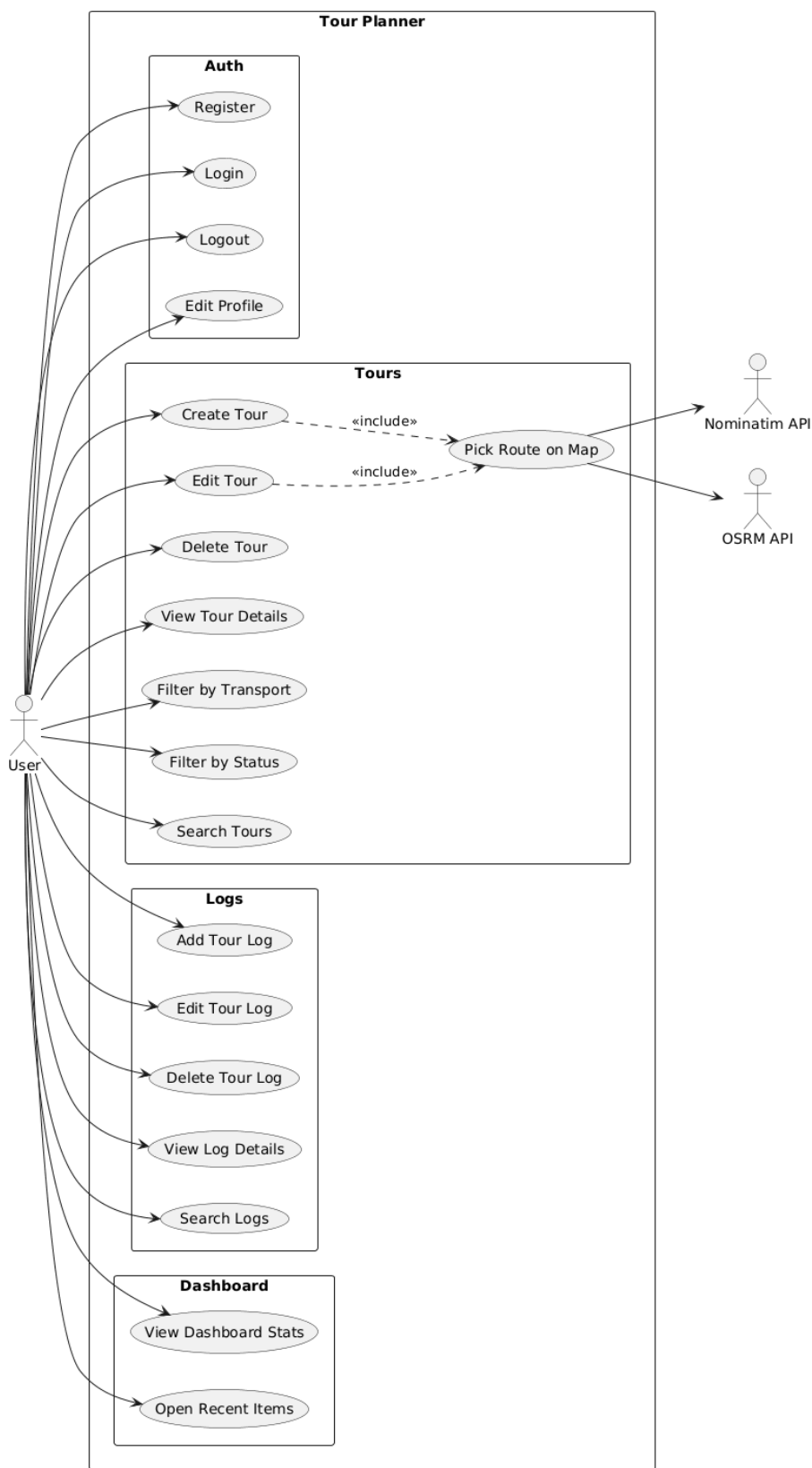
Reasoning. The session focused on UX, MVVM correctness and search/responsiveness. The persistence layer (EF Core + Postgres) and JWT-style authentication are explicitly marked as the next milestone in the source (TODO: wire backend ... in AuthService).

3 Design Patterns Implemented

Pattern	Where	Why
MVVM	viewmodels/*.viewmodel.ts consumed by eachcomponent	Separate stateful logicfrom views; testable VMs
Singleton	All services with providedIn: 'root'	Single source of truth fortours, logs, search, layout
Observer /Reactive	Angular signals + computed()	Push-based reactivitywithout manualsubscriptions
Facade	ModalService, LayoutService	Hide modal/layoutcoordination behind oneAPI
Adapter	NominatimService, OsrRoutingService	Decouple Leaflet/Pickerfrom external HTTP APIs
Strategy	OsrRoutingService.route(co ords,transportType) selects the OSRM profile (foot /bike / car)	Routing varies pertransport without ifbranches in callers

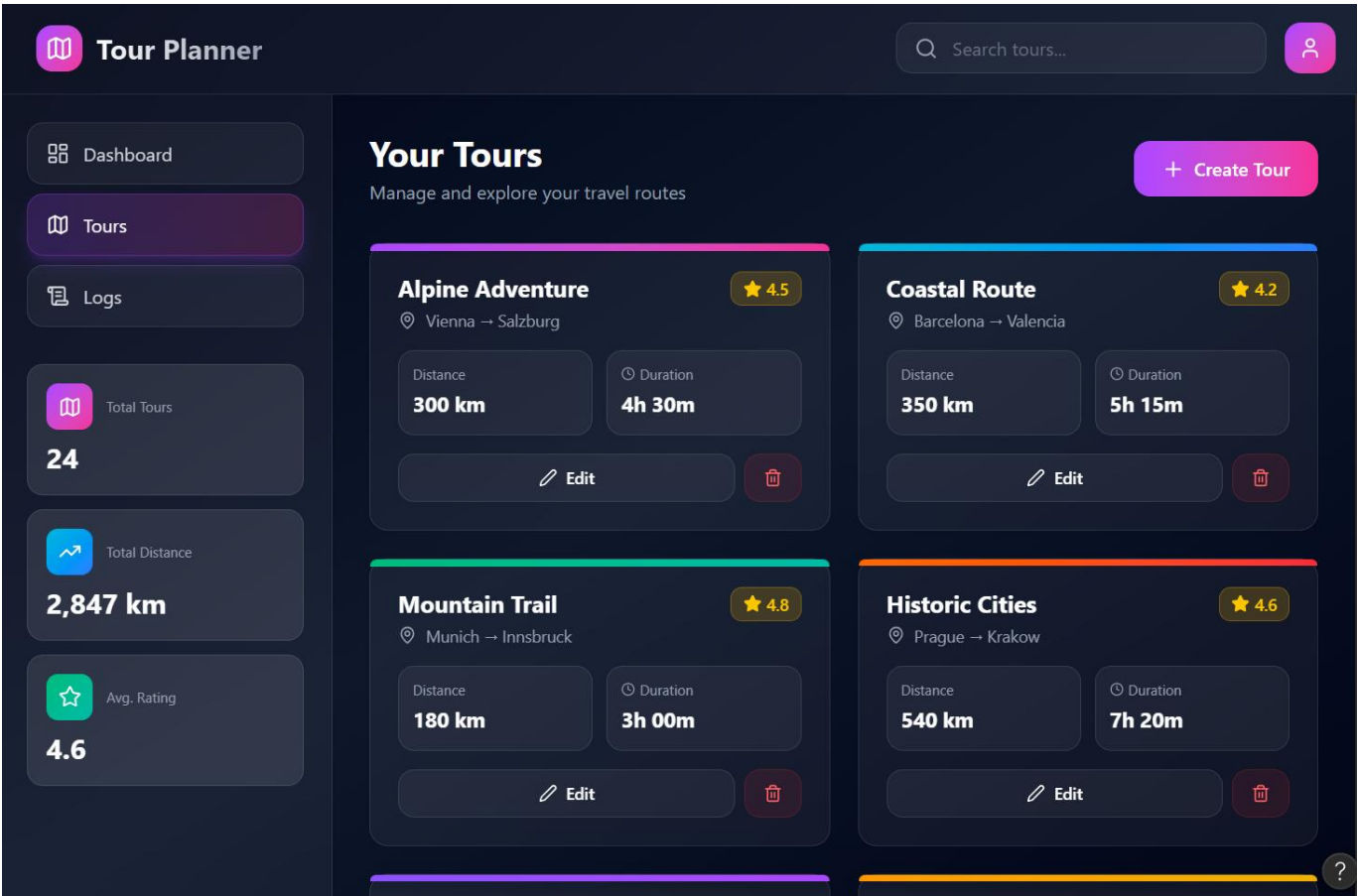
The **Adapter pattern** is the most prominent one: the entire map picker, create-tour and tour-detail screens are written against the domain types SearchResult and RouteResult. Swappingthe provider (e.g. Mapbox) would mean replacing two service implementations only.

4 Use-Case Diagram



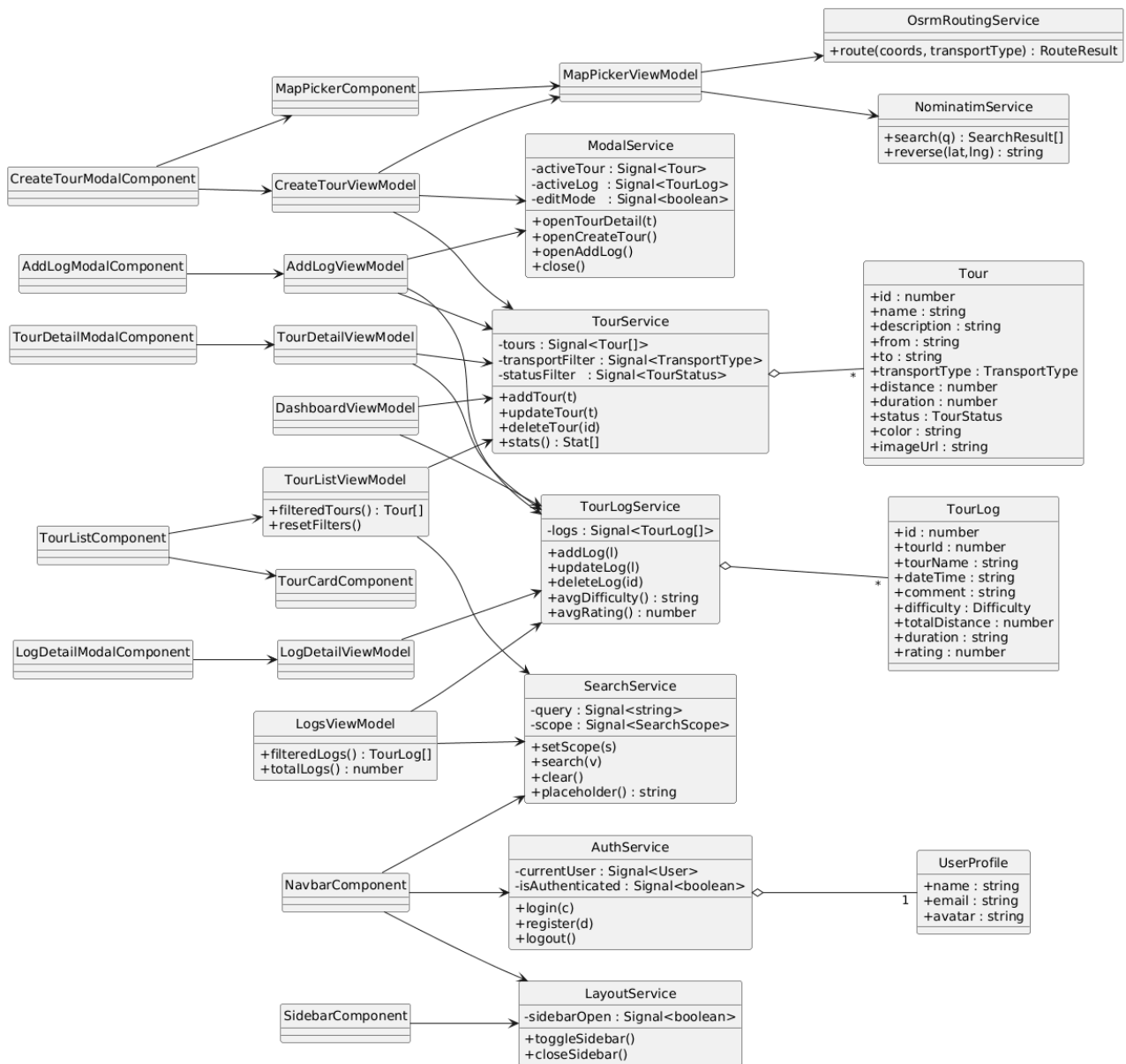
5 UI-Flow Wireframes

Figma-Link: [Tour List Dashboard Wireframe – Figma Make](#)

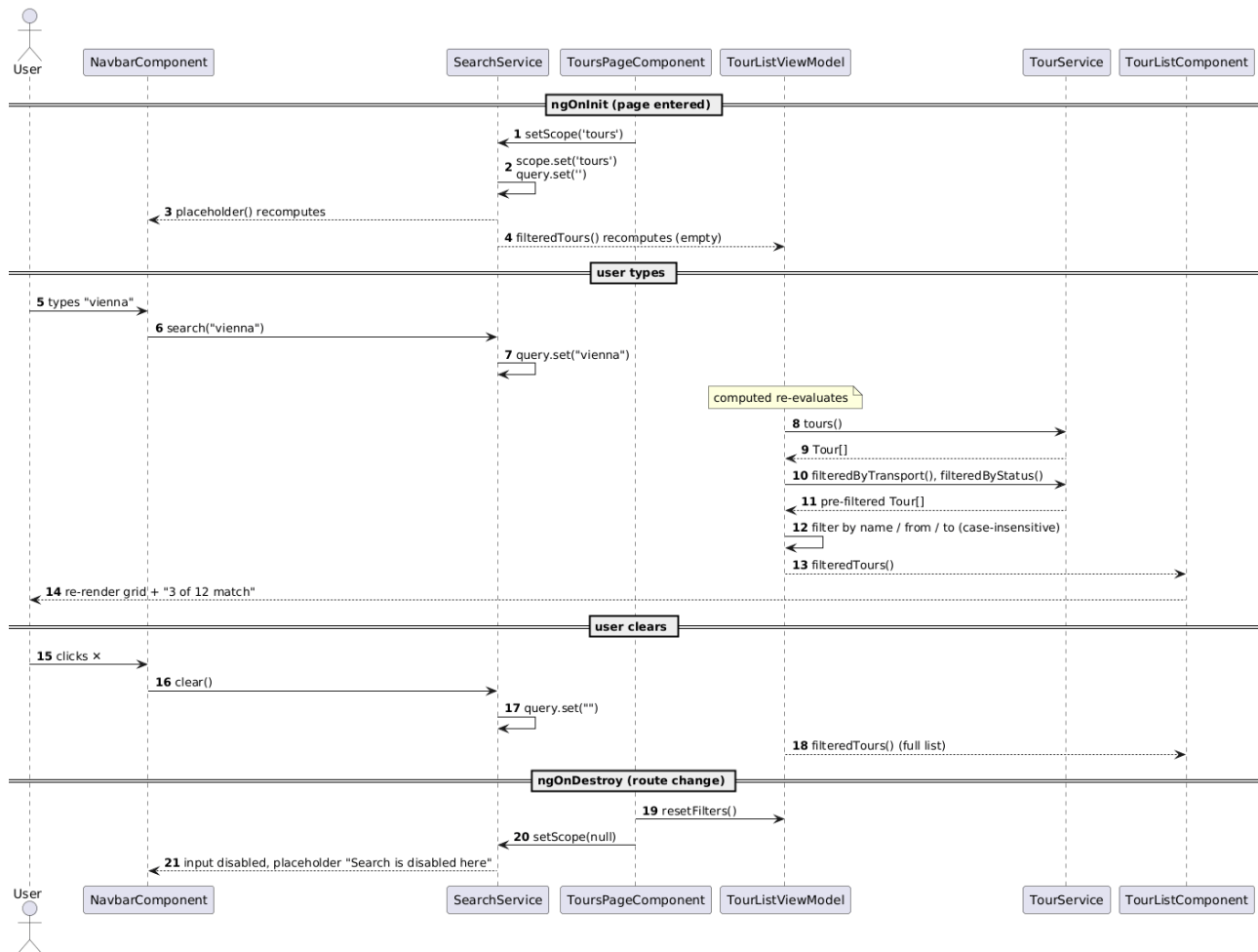


Created with integrated AI

6 Architecture — Class Diagram



7 Sequence Diagram — Full-text Search on Tours



8 Time Tracking

Time was tracked through the Git history (47 commits across 9 active days, **April 28 → June 3, 2026**). Each day is estimated by the volume of commits and the size of the resulting diffs.

Date	Focus	Commits	Hours
2026-04-28	Initial repository, scaffold backend & frontend	2	1.5
2026-04-29	README baseline	1	0.5
Date	Focus	Commits	Hours
2026-04-30	UML drafts (use case, class)	3	2.0
2026-05-06	First Angular implementation, tours/logs components, dashboard, filters, MVVM cleanup	9	7.0
2026-05-07	Tour model cleanup (status added, rating moved to logs), sidebar	1	2.0
2026-05-29	Remove template favicon	1	0.5
2026-05-30	Map integration: Leaflet, Nominatim search, OSRM routing pertransport	4	4.5
2026-06-02	Map UX (zoom, fullscreen, cursors), SSR guard, MVVM	8	6.5

	refactor of picker, tour validation, demo data, delete confirm		
2026-06-03	Auth page, logout, tour log management (validation + inherited distance/duration), search UX (scope, clear, match-count), responsive layout (mobile drawer, modal grids)	18	9.5
Total		47	≈ 34h

Per-area split (rough): map & routing ≈ 9 h · MVVM/refactors ≈ 6 h · tour & log CRUD ≈ 8 h · search UX ≈ 3 h · responsive layout ≈ 3 h · UML & docs ≈ 2.5 h · setup ≈ 2.5 h.

9 Repository MapTour

The Git history (see `git log --oneline --all`) documents every step: SCRUM-10 (TourManagement), SCRUM-11 (Tour Log Management), SCRUM-26 (Map), SCRUM-33 (SearchImprovement), plus the Auth page and the Better GUI / responsive pass. As required, the history is part of the documentation and is not duplicated here.